Documentação Técnica Profissional — WebSec Scanner (CSIRT Edition)

1. Introdução

A documentação técnica de software é um elemento fundamental para garantir a eficiência, a segurança e a sustentabilidade de projetos de tecnologia, especialmente em ambientes críticos como os de resposta a incidentes de segurança (CSIRT). Este documento apresenta a versão aprimorada da documentação do WebSec Scanner (CSIRT Edition), uma ferramenta dedicada à detecção automatizada de vulnerabilidades em aplicações web, com foco em ambientes de resposta a incidentes, conformidade regulatória e integração com inteligência de ameaças. O texto foi reestruturado para maximizar a clareza, a coesão e o profissionalismo, alinhando-se às melhores práticas internacionais de documentação técnica e segurança operacional.

A seguir, são detalhados os principais tópicos: visão geral do sistema, segurança operacional (OPSEC), arquitetura e fluxo de execução, integração com inteligência de ameaças (VirusTotal), geração de relatórios, conformidade legal, mitigação de falsos positivos, ambientes de teste recomendados, segurança de desenvolvimento, integração com pipelines CI/CD, preservação de evidências e métricas de desempenho.

2. Visão Geral do Sistema

2.1 Objetivo e Escopo

O WebSec Scanner (CSIRT Edition) é uma ferramenta educacional e operacional para detecção automatizada de vulnerabilidades em aplicações web. Seu propósito é apoiar equipes de resposta a incidentes (CSIRTs), profissionais de segurança ofensiva e times de DevSecOps na identificação, análise e documentação de falhas de segurança em ambientes autorizados.

A ferramenta foi projetada para:

- Realizar varredura automatizada (crawling) de aplicações web.
- Identificar pontos de entrada (parâmetros, formulários) e testá-los com payloads específicos para diferentes classes de vulnerabilidades.
- Gerar relatórios técnicos e executivos em múltiplos formatos (HTML, JSON).

- Integrar-se a plataformas de inteligência de ameaças para enriquecimento de indicadores.
- Atender aos requisitos operacionais e legais de CSIRTs, incluindo preservação de evidências e conformidade com a LGPD.

2.2 Público-Alvo

A documentação é destinada a:

- Analistas e gestores de CSIRT.
- Profissionais de segurança ofensiva e defensiva.
- Desenvolvedores e engenheiros de DevSecOps.
- Auditores de conformidade e privacidade.
- Equipes de governança, risco e compliance (GRC).
-

2.3 Funcionalidades Principais

- **Crawling recursivo** com controle de profundidade.
- **Detecção e extração de formulários** e parâmetros.
- **Testes automatizados** para múltiplas classes de vulnerabilidades (SQLi, XSS, LFI, Command Injection).
- **Gestão modular de payloads**.
- **Geração de relatórios** em HTML e JSON, com sumário executivo.
- **Integração com VirusTotal** para enriquecimento de indicadores.
- **Logging detalhado** para auditoria e forense.
- **Configuração flexível** via arquivos e parâmetros de linha de comando.
- **Isolamento de execução** via containers (Docker).
- **Suporte a ambientes de teste seguros** (DVWA, WebGoat, bWAPP).

3. Segurança Operacional (OPSEC)

3.1 Princípios de Segurança

A operação de scanners de vulnerabilidade exige rigorosos controles de segurança operacional (OPSEC) para evitar exposição indevida, vazamento de informações sensíveis e impactos não intencionais em ambientes de produção. O WebSec Scanner adota os seguintes princípios:

- **Princípio do menor privilégio**: execução com permissões mínimas necessárias.
- **Isolamento de ambiente**: uso de containers Docker para sandboxing.
- **Gestão segura de credenciais**: integração com Vault e rotação periódica de segredos.

- **Registro e auditoria**: logs detalhados e trilhas de auditoria para todas as ações.
- **Conformidade com a LGPD e normativos de privacidade**: anonimização de dados sensíveis, retenção controlada de evidências e políticas de descarte seguro.

3.2 Procedimentos Operacionais Padrão (POPs)

Para garantir a segurança e a repetibilidade das operações, recomenda-se a adoção de Procedimentos Operacionais Padrão (POPs) e runbooks, incluindo:

- **Checklist pré-execução**: validação de escopo, autorização formal, definição de limites e contatos de emergência.
- **Execução controlada**: uso de ambientes de teste, limitação de taxa de requisições (rate limiting), monitoramento de recursos.
- **Gestão de incidentes**: plano de resposta para falhas, interrupções ou detecção de atividades suspeitas.
- **Preservação de evidências**: coleta, armazenamento e documentação conforme cadeia de custódia digital.

3.3 Isolamento e Sandboxing

A execução do scanner deve ocorrer em ambientes isolados, preferencialmente containers Docker, para mitigar riscos de comprometimento do host e facilitar a reprodutibilidade dos testes. Recomenda-se:

- **Criação de redes virtuais dedicadas** para containers.
- **Restrição de acesso à internet** durante testes sensíveis.
- **Monitoramento de recursos** (CPU, memória, disco) para evitar negação de serviço acidental.

3.4 Gestão de Credenciais e Segredos

- **Armazenamento seguro** de segredos via HashiCorp Vault ou AWS Secrets Manager.
- **Evitar hardcoding** de senhas e tokens em código-fonte.
- **Rotação periódica** de credenciais e tokens de API.
- **Uso de autenticação forte** (MFA, OAuth 2.0, OpenID Connect) para integrações sensíveis.

Arquitetura e Fluxo de Execução

4.1 Arquitetura Modular O WebSec Scanner foi desenvolvido em Python 3.8+, com arquitetura modular e extensível, composta pelos seguintes componentes principais: • Módulo de crawling: responsável pela navegação recursiva e descoberta de URLs e formulários. • Módulo de análise de entrada: identifica parâmetros GET/POST e campos de formulários. • Módulo de payloads: gerencia e aplica payloads específicos para cada classe de vulnerabilidade. • Módulo de testes de vulnerabilidade: executa ataques controlados e analisa respostas. • Módulo de geração de relatórios: consolida achados e produz relatórios em múltiplos formatos. • Módulo de integração com inteligência de ameaças: consulta e submete indicadores a plataformas externas (ex: VirusTotal). • Módulo de logging e auditoria: registra eventos, erros e evidências para análise forense.

4.2 Fluxo de Execução O fluxo operacional do WebSec Scanner segue as etapas abaixo:

1. Inicialização e configuração:
   • Carregamento de parâmetros (URL alvo, profundidade, timeout, diretório de saída, delay entre requisições). • Validação de ambiente e permissões.
2. Crawling e descoberta:
   • Navegação recursiva a partir da URL alvo, respeitando a profundidade definida. • Identificação de formulários, parâmetros e pontos de entrada.
3. Teste de vulnerabilidades:
   • Aplicação de payloads específicos para cada classe de vulnerabilidade (SQLi, XSS, LFI, Command Injection). • Análise de respostas HTTP, códigos de status, mensagens de erro e comportamentos anômalos. • Detecção de headers de segurança ausentes.

4. Enriquecimento de indicadores:
   • Consulta de hashes, URLs e domínios suspeitos em plataformas de threat intelligence (ex: VirusTotal). • Submissão opcional de amostras para análise comportamental.
5. Geração de relatórios:
   • Consolidação dos achados em relatórios HTML e JSON. • Inclusão de sumário executivo, métricas (MTTD, MTTR, severidade), evidências e recomendações.
6. Finalização e limpeza:
   • Armazenamento seguro de logs e evidências. • Descarte seguro de dados temporários e segredos.

4.3 Gestão de Payloads Os payloads

são armazenados em arquivos de texto organizados por tipo de vulnerabilidade (ex: , , ,). Cada linha representa um payload distinto, permitindo fácil atualização e

customização. • Adição de novos payloads: basta editar os arquivos correspondentes. • Suporte a encoding: payloads podem ser automaticamente codificados (URL, Base64, Hex, HTML entities) para evasão de filtros. • Obfuscação e evasão de WAF: técnicas opcionais de fragmentação, inserção de comentários e variação de case.

4.4 Técnicas de Detecção O scanner implementa técnicas clássicas e modernas para detecção de vulnerabilidades, incluindo: • SQL Injection: union-based, error-based, boolean-based, time-based, stacked queries. • Cross-Site Scripting (XSS): reflected, stored, DOM-based, blind. • Local File Inclusion (LFI): path traversal, bypass de filtros, exploração de wrappers. • Command Injection: injeção de comandos em sistemas Linux/Windows, uso de operadores (, , , backticks). • Detecção de headers de segurança ausentes: X-Frame-Options, Content-Security-Policy, Strict-Transport-Security, entre outros.

4.5 Performance, Escalabilidade e Limites

- **Controle de concorrência**: limite configurável de threads/processos para evitar sobrecarga do alvo.
- **Rate limiting**: delay entre requisições para evitar bloqueios e detecção por WAF.
- **Timeouts configuráveis**: prevenção de travamentos em respostas lentas.
- **Monitoramento de recursos**: logs de uso de CPU, memória e tempo de execução.

Integração com Inteligência de Ameaças (VirusTotal)

5.1 Objetivo da Integração A integração com plataformas de inteligência de ameaças, como o VirusTotal, permite enriquecer indicadores coletados durante o scan (hashes, URLs, domínios, IPs) com informações de reputação, detecção por múltiplos motores antivírus e análise comportamental (Code Insight). 5.2 Funcionalidades Suportadas • Consulta de reputação: verificação de hashes, URLs, domínios e IPs contra bases de dados de ameaças conhecidas. • Submissão de amostras: envio opcional de arquivos ou URLs suspeitos para análise dinâmica. • Análise comportamental (Code Insight): detecção de padrões suspeitos em código, execução silenciosa de comandos, tentativas de escalonamento de privilégios e coleta de credenciais. • Pivotamento de

IoCs: identificação de relações entre indicadores (ex: arquivos associados a domínios maliciosos).

5.3 Fluxo de Integração

1. Extração de indicadores: durante o scan, hashes de arquivos, URLs e domínios suspeitos são coletados.

2. Consulta à API do VirusTotal: • Uso de chave de API (preferencialmente armazenada em Vault). • Respeito aos limites de requisições (4/minuto na API pública, 500/dia).
3. Análise dos resultados: • Classificação de risco (High, Medium, Low, ZeroRisk). • Detalhamento de detecções por motores antivírus. • Análise de comportamento e relações (sandbox, MITRE ATT&CK).
4. Enriquecimento dos relatórios: inclusão dos resultados de reputação e análise comportamental nos relatórios técnicos.

5.4 Considerações Operacionais

- **Armazenamento seguro da chave de API**: nunca expor em código ou logs.
- **Respeito à privacidade e conformidade**: submissão de amostras apenas com autorização explícita.
- **Limitações da API pública**: considerar upgrade para API premium em ambientes corporativos.
- **Automação e integração com SIEM/SOAR**: possibilidade de envio automático de indicadores enriquecidos para plataformas de orquestração e resposta a incidentes.

6. Geração de Relatórios

6.1 Tipos de Relatórios

O WebSec Scanner suporta múltiplos formatos de relatório, adaptados a diferentes públicos e finalidades:

- **Relatório Técnico (HTML)**: detalhado, com evidências, payloads utilizados, respostas HTTP, capturas de tela e recomendações técnicas.
- **Relatório Estruturado (JSON)**: adequado para integração com SIEM, SOAR e outras ferramentas de automação.

- **Sumário Executivo**: visão de alto nível para gestores, destacando principais achados, severidade, riscos e recomendações prioritárias.

6.2 Estrutura Recomendada

Com base nas melhores práticas e modelos internacionais, recomenda-se a seguinte estrutura:

1. **Capa e Sumário**
2. **Resumo Executivo**
    a. Escopo do teste
    b. Principais descobertas
    c. Nível de risco geral
    d. Conclusão e próximos passos
3. **Metodologia**
    a. Ferramentas e técnicas utilizadas
    b. Limitações e exclusões
4. **Resultados Detalhados**
    a. Vulnerabilidades identificadas (por tipo, local, parâmetro)
    b. Evidências (logs, capturas de tela, trechos de código)
    c. Análise de risco (CVSS, severidade, impacto)
    d. Recomendações de mitigação

- **Métricas e Indicadores**

    - MTTD (Mean Time to Detect)
    - MTTR (Mean Time to Respond)
    - Taxa de falsos positivos
    - Cobertura de detecção

4. **Anexos**

- Logs completos
- Payloads utilizados
- Evidências brutas
- Cadeia de custódia digital

6.3 Métricas e Indicadores

- **MTTD (Mean Time to Detect)**: tempo médio para detecção de vulnerabilidades.
- **MTTR (Mean Time to Respond)**: tempo médio para resposta e correção.

- **Severidade**: classificação baseada em CVSS ou critérios próprios (Crítico, Alto, Médio, Baixo).
- **Taxa de falsos positivos**: percentual de achados não confirmados.
- **Cobertura de detecção**: percentual de superfícies de ataque testadas.

6.4 Boas Práticas de Comunicação

- **Clareza e objetividade**: evitar jargões excessivos, explicar termos técnicos.
- **Estrutura lógica**: seções bem definidas, sumário, links internos.
- **Visualização de dados**: uso de tabelas, gráficos e diagramas para facilitar a compreensão.
- **Validação interna**: revisão por pares antes da entrega.
- **Atualização contínua**: revisão periódica dos modelos e templates.

7. Conformidade Legal e Privacidade

7.1 LGPD e Proteção de Dados

A ferramenta e seus operadores devem observar rigorosamente a Lei Geral de Proteção de Dados (LGPD) e demais normativos aplicáveis:

- **Coleta mínima de dados pessoais**: limitar a coleta ao estritamente necessário.
- **Anonimização e pseudonimização**: sempre que possível, remover identificadores pessoais de logs e evidências.
- **Retenção controlada**: definir prazos de retenção e políticas de descarte seguro.
- **Consentimento e autorização**: executar testes apenas em ambientes e sistemas com autorização formal e explícita.
- **Preservação da cadeia de custódia**: garantir integridade, autenticidade e rastreabilidade das evidências digitais.

7.2 Avisos Legais e Uso Autorizado

- **Termo de uso**: a ferramenta deve ser utilizada exclusivamente para fins educacionais ou em ambientes autorizados.
- **Responsabilidade**: o operador é responsável por obter todas as autorizações necessárias e respeitar os limites do escopo definido.
- **Proibição de uso malicioso**: uso não autorizado pode configurar crime conforme legislação vigente (Lei 12.737/2012, Marco Civil da Internet).

- **Documentação de escopo e autorização**: manter registros detalhados de contratos, termos de autorização e NDAs específicos para pentest.

7.3 Preservação de Evidências e Logs

- **Cópia forense**: sempre trabalhar com cópias, nunca com o original.
- **Documentação detalhada**: registrar data, hora, operador, ferramentas e condições de coleta.
- **Hashing e integridade**: calcular e registrar hashes criptográficos de evidências.
- **Armazenamento seguro**: uso de cofre digital, acesso restrito e controle de acesso.
- **Descarte seguro**: eliminação definitiva após o prazo de retenção ou encerramento do caso.

8. Mitigação de Falsos Positivos e Validação de Achados

8.1 Desafios dos Falsos Positivos

Scanners automatizados frequentemente geram falsos positivos, que podem sobrecarregar equipes de resposta e comprometer a credibilidade dos relatórios. As principais causas incluem:

- Regras de detecção genéricas.
- Ambientes complexos com comportamentos legítimos atípicos.
- Falta de contexto sobre o funcionamento da aplicação.

8.2 Estratégias de Redução

- **Machine learning e análise comportamental**: uso de algoritmos para diferenciar padrões legítimos de ataques reais.
- **Regras de correlação personalizadas**: ajuste das regras de detecção ao contexto da organização.
- **Enriquecimento de alertas com contexto**: inclusão de informações sobre ativos, localização, perfil do usuário.
- **Testes e auditorias regulares**: revisão periódica das regras e simulação de ataques controlados.
- **Priorização por criticidade**: classificação dos alertas com base no impacto potencial.

- **Automação de respostas para incidentes comuns**: playbooks automatizados para triagem inicial.

8.3 Validação Manual

- **Reprodução controlada**: tentativa de reproduzir manualmente os achados mais críticos.
- **Coleta de evidências adicionais**: logs, capturas de tela, respostas HTTP detalhadas.
- **Consulta a especialistas**: revisão por analistas experientes em casos ambíguos.
- **Feedback contínuo**: ciclo de melhoria baseado em incidentes reais e feedback das equipes operacionais.

9. Ambientes de Teste Recomendados

9.1 Importância do Ambiente Controlado

Testes de vulnerabilidade devem ser realizados, preferencialmente, em ambientes controlados e isolados, para evitar impactos em sistemas de produção e garantir a legalidade das operações.

9.2 Plataformas Sugeridas

- **OWASP DVWA (Damn Vulnerable Web Application)**: aplicação PHP intencionalmente vulnerável para prática de ataques web.
- **OWASP WebGoat**: ambiente Java com múltiplos cenários de vulnerabilidades.
- **bWAPP (Buggy Web Application)**: aplicação PHP/MySQL com mais de 100 vulnerabilidades simuladas.
- **Metasploitable2**: máquina virtual Linux vulnerável para testes de exploração.
- **Ambientes Docker customizados**: containers isolados para simulação de cenários específicos.

9.3 Boas Práticas

- **Isolamento de rede**: evitar exposição dos ambientes de teste à internet.
- **Documentação do ambiente**: registrar configurações, versões e snapshots para reprodutibilidade.
- **Atualização periódica**: manter os ambientes alinhados com as últimas versões e cenários de ataque.

10. Segurança de Desenvolvimento e DevSecOps

10.1 Princípios de Desenvolvimento Seguro

- **Security by Design**: segurança incorporada desde a concepção do software.
- **Privacy by Design**: minimização da exposição de dados pessoais em todos os fluxos.
- **Gestão de dependências**: monitoramento contínuo de bibliotecas e componentes de terceiros para evitar riscos de cadeia de suprimentos.
- **Análise estática e dinâmica**: uso de SAST (Static Application Security Testing) e DAST (Dynamic Application Security Testing) integrados ao pipeline CI/CD.
- **Gestão de segredos**: uso de Vault, rotação periódica e controle de acesso granular.

10.2 Integração com CI/CD e DevSecOps

- **Automação de testes de segurança**: integração do scanner em pipelines de CI/CD para detecção precoce de vulnerabilidades.
- **Bloqueio de builds críticos**: impedir deploys em produção caso vulnerabilidades críticas sejam detectadas.
- **Monitoramento contínuo**: uso de ferramentas de observabilidade e correlação de eventos de segurança.
- **Feedback rápido**: alertas automáticos para desenvolvedores em caso de falhas.

10.3 Ferramentas Complementares

- **Burp Suite**: proxy para testes manuais e automação de ataques web.
- **OWASP ZAP**: scanner open source para integração em pipelines.
- **sqlmap**: automação de testes de SQL Injection.
- **Nmap**: mapeamento de redes e detecção de serviços.
- **Trivy, Clair, Grype**: scanners de vulnerabilidades em containers e imagens Docker.

11. Preservação de Evidências e Logs para Auditoria Forense

11.1 Cadeia de Custódia Digital

A preservação adequada de evidências digitais é essencial para garantir a validade jurídica dos achados e possibilitar auditorias e investigações forenses.

- **Cópia forense**: sempre trabalhar com cópias bit-a-bit dos dados originais.
- **Documentação detalhada**: registro de todas as etapas, ferramentas e operadores envolvidos.
- **Hashing**: cálculo de hashes (MD5, SHA-256) para garantir integridade.
- **Armazenamento seguro**: uso de cofre digital, acesso restrito e logs de acesso.
- **Descarte controlado**: eliminação definitiva após encerramento do caso ou prazo legal.

11.2 Logs e Auditoria

- **Registro de eventos**: logs detalhados de todas as ações, comandos, payloads e respostas.
- **Retenção conforme política**: definição de prazos de retenção alinhados à LGPD e normativos internos.
- **Anonimização de dados sensíveis**: remoção de identificadores pessoais antes do armazenamento ou compartilhamento.
- **Auditoria periódica**: revisão dos logs e evidências por equipe independente.

12. Performance, Escalabilidade e Limites

12.1 Controle de Concorrência e Rate Limiting

- **Configuração de threads/processos**: ajuste conforme capacidade do ambiente e do alvo.
- **Delay entre requisições**: prevenção de bloqueios por WAF e sobrecarga do sistema alvo.
- **Timeouts**: definição de limites para evitar travamentos.

12.2 Monitoramento de Recursos

- **Uso de CPU, memória e disco**: logs de consumo durante a execução.
- **Alertas automáticos**: notificação em caso de uso excessivo ou falhas críticas.

- **Escalabilidade horizontal**: possibilidade de execução distribuída em múltiplos containers ou hosts.

13. Riscos de Cadeia de Suprimentos e Dependências Transitivas

- **Monitoramento contínuo de dependências**: uso de ferramentas como OWASP Dependency-Check.
- **Avaliação de riscos de terceiros**: análise de fornecedores de bibliotecas e componentes.
- **Política de atualização**: atualização periódica de dependências e aplicação de patches de segurança.
- **Gestão de vulnerabilidades em containers**: uso de scanners específicos para imagens Docker.

14. Avisos Legais, Uso Autorizado e Políticas de Responsabilidade

- **Uso restrito**: a ferramenta deve ser utilizada apenas em ambientes autorizados e para fins educacionais ou de segurança.
- **Responsabilidade do operador**: obtenção de autorizações formais, definição clara de escopo e limites.
- **Proibição de uso malicioso**: qualquer uso fora dos limites autorizados pode configurar crime.
- **Documentação de escopo e autorização**: manutenção de registros detalhados para proteção jurídica.

15. Modelos de Relatório e Comunicação com Stakeholders

15.1 Estrutura Recomendada

- **Resumo Executivo**: visão geral dos achados e recomendações prioritárias.

- **Metodologia**: detalhamento das técnicas, ferramentas e limitações.
- **Resultados Detalhados**: descrição das vulnerabilidades, evidências e análise de risco.
- **Recomendações**: ações corretivas e preventivas.
- **Métricas e Indicadores**: MTTD, MTTR, severidade, taxa de falsos positivos.
- **Anexos**: logs, payloads, evidências brutas, cadeia de custódia.

15.2 Boas Práticas

- **Clareza e objetividade**: linguagem acessível, explicação de termos técnicos.
- **Visualização de dados**: uso de tabelas, gráficos e diagramas.
- **Validação interna**: revisão por pares antes da entrega.
- **Atualização contínua**: revisão periódica dos modelos e templates.

16. Padrões e Frameworks Aplicáveis

- **NIST Cybersecurity Framework (CSF) 2.0**: referência para gestão de riscos de segurança cibernética.
- **ISO/IEC 27001**: requisitos para sistemas de gestão de segurança da informação.
- **OWASP Top 10**: principais vulnerabilidades em aplicações web.
- **FIRST CSIRT Services Framework**: padrões para operação e cooperação entre CSIRTs.
- **CERT.br, TLP, SIM3**: padrões nacionais e internacionais para resposta a incidentes.

17. Considerações Finais

A documentação técnica aprimorada do WebSec Scanner (CSIRT Edition) foi estruturada para atender aos mais altos padrões de clareza, coesão e profissionalismo, alinhando-se às melhores práticas internacionais de segurança, privacidade e conformidade. O documento cobre todos os aspectos críticos para a operação segura, eficiente e legal da ferramenta, incluindo arquitetura, OPSEC, integração com inteligência de ameaças, geração de relatórios, mitigação de falsos positivos, preservação de evidências e comunicação com stakeholders.

A adoção rigorosa dessas diretrizes e procedimentos é fundamental para garantir a eficácia dos testes de segurança, a proteção dos ativos digitais e a conformidade com os requisitos legais e regulatórios. Recomenda-se a revisão periódica desta documentação, a atualização contínua dos modelos e a capacitação das equipes envolvidas, promovendo uma cultura de segurança, colaboração e melhoria contínua.

Nota: Esta documentação foi elaborada com base em referências técnicas, normativas e operacionais atualizadas, incluindo guias do NIST, OWASP, ISO/IEC, CERT.br, FIRST, LGPD, além de práticas recomendadas de DevSecOps, forense digital e gestão de vulnerabilidades. Para dúvidas, sugestões ou contribuições, recomenda-se o uso de sistemas de controle de versão e colaboração (ex: GitHub, Confluence) para garantir rastreabilidade e atualização contínua dos conteúdos.